



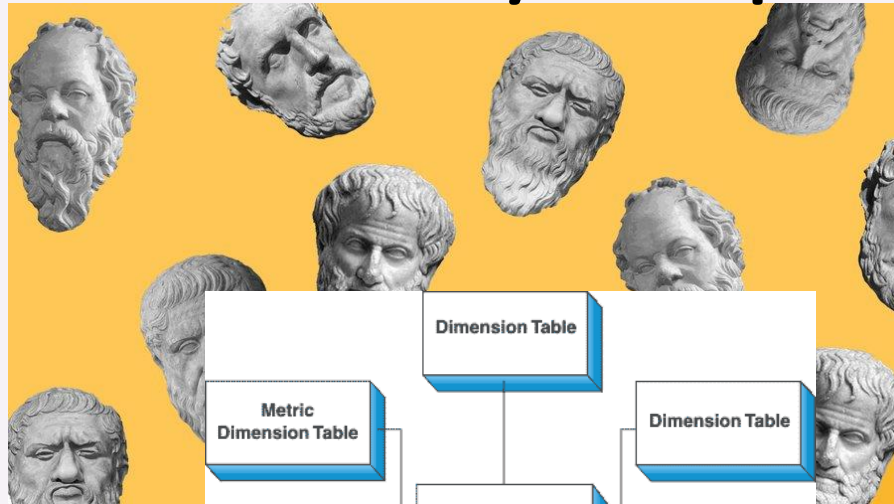
План: Зачем нужен DAX?

DAX – Data Analysis Expressions (выражения анализа данных)



Язык формул, который используется в Power BI, а также в других продуктах Microsoft: Power Pivot (надстройка Excel) и табличных моделях SSAS

Минутка философии: Зачем нужен DAX?

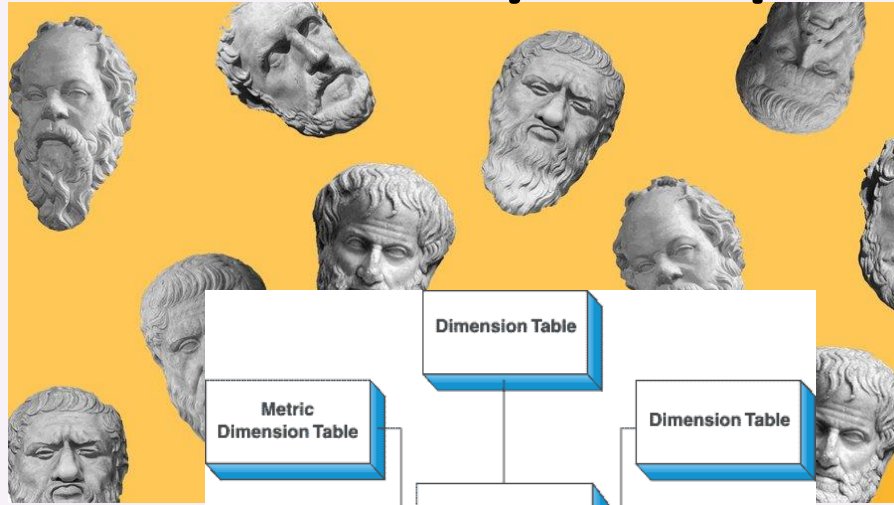


Минутка философии: Зачем нужен DAX?

Не для базовой аналитики: базовая агрегацию можно и в Excel сделать



Минутка философии: Зачем нужен DAX?

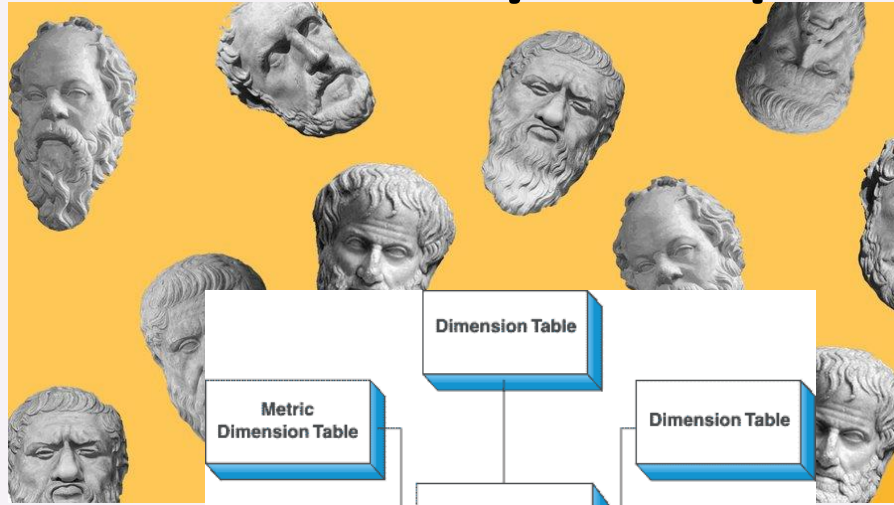


Не для базовой аналитики: базовая агрегацию можно и в Excel сделать

Для сложной аналитики:
предварительные вычисления/pivot
tables + базовая агрегация =
получение *информации*



Минутка философии: Зачем нужен DAX?



Не для базовой аналитики: базовая агрегацию можно и в Excel сделать

Для сложной аналитики:
предварительные вычисления/pivot
tables + базовая агрегация =
получение **информации**



Блок за блоком строим как
конструктор формулу DAX и в конце
делаем базовую агрегацию

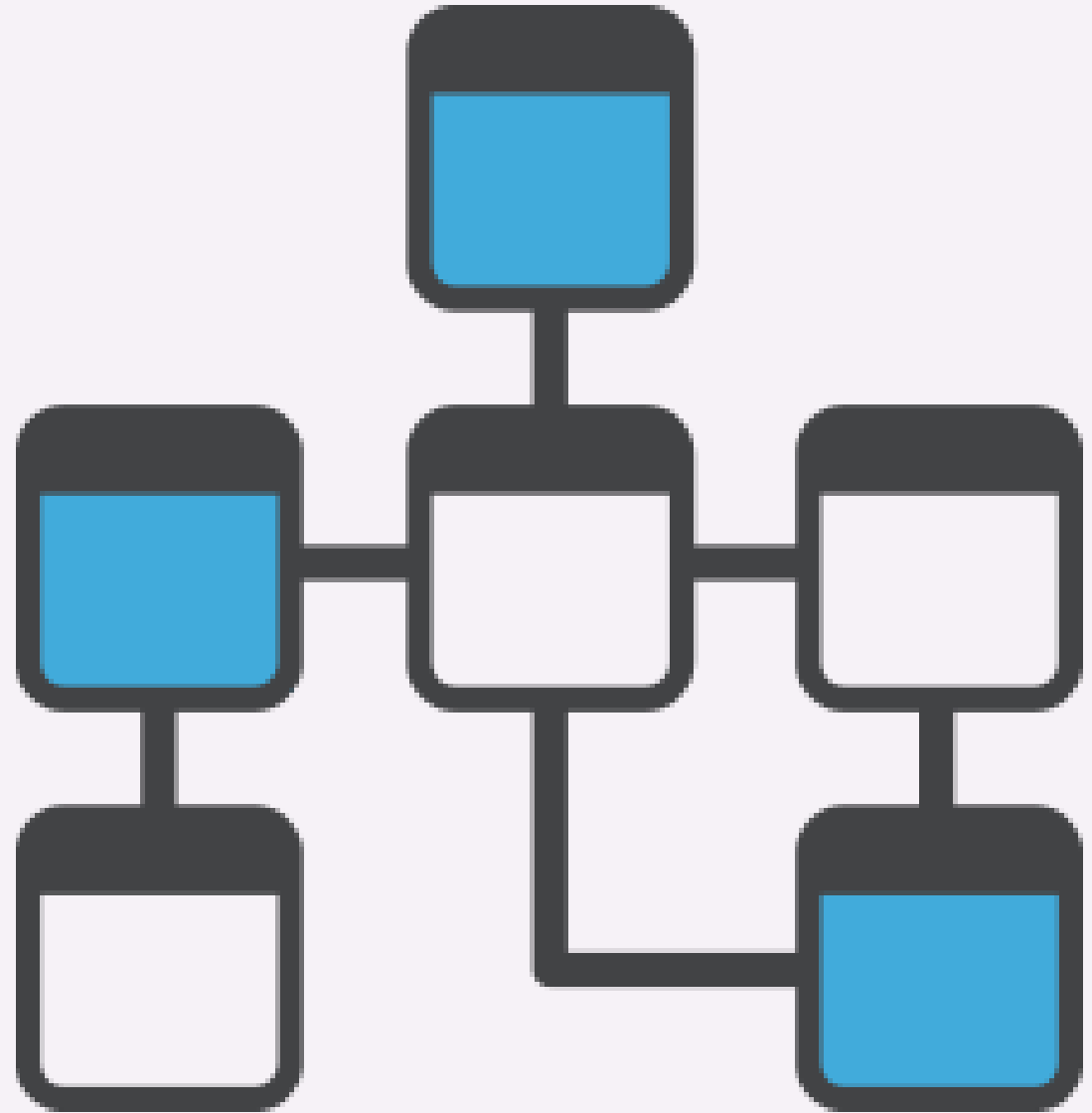
Конструктор формул DAX:

1. Фильтруем табл + базовая агрегация = информация
2. Фильтруем + расчет + базовая агрегация = информация
3. Фильтруем + предварительная агрегация + базовая агрегация = информация
4. Фильтруем + изменение контекста + базовая агрегация = информация
5. Комбинация 1-4



Модель данных – основа отчета

*Нужно подходить к ней
максимально
ответственно, т.к.
неправильная организация
данных может привести к
некорректному
отображению данных в
ваших визуалах и сильно
усложнить вычисления*



Как правильно организовать данные?

- Подключиться ко всем источникам в PowerQuery, параметрами, Enable Load, другими способами ограничить датасет и в модель загрузить только нужное.
- Минусы: сложно поддерживать, не все источники «дают» данные (ограничения по кол-ву МБ, по датам, и тп), ограничение по объему датасета (1 GB limit per dataset, PBIX не может быть опубликован в облако если более 1 GB)
- Решение: хранилище данных (data warehouse), это может быть SQL база данных на самых первых этапах. В DWH храним все в самом детальном виде, в модель подгружаем только нужное, агрегаты

Два вида функций DAX:

- **Scalar (скалярные)** – возвращают одно значение (тип данных может быть при этом любой, Date, Numeric, Text и тд.) , эти функции могут быть использованы для создания Мер и Колонок
 - Большинство Aggregation Functions (SUM, AVERAGE, etc)
 - X-Aggregation Functions (SUMX, AVERAGEX, etc)
 - Date Functions (Year, Month, etc)
 - Calculate, Related
 - ...
- **Tabular (табличные)** – возвращают таблицу, могут быть использованы для создания Таблиц
 - ALL, FILTER, ADDCOLUMNS
 - RelatedTable
 - SamePeriodLastYear
 - Summarize
 - ...
- Два вида могут быть скомбинированы (конструктор) для разных целей
- Исключения: CrossFilter

Aggregation functions

- Работают только с колонками, содержащими числа
- Агрегируют только ОДНУ колонку
 - SUM (Sales[Price]) -- OK
 - SUM (Sales[Price] * Sales[Quantity]) – НЕ OK
- SUM – работает с 1 столбцом
- MAX / MIN – работают с несколькими столбцами
- AVERAGE – работает с 1 столбцом
- COUNT – работает с 1 столбцом
- COUNTBLANK – работает с 1 столбцом
- DISTINCTCOUNT – работает с 1 столбцом
- COUNTROWS – работает с таблицей

X-Aggregation functions (Итераторы)

- Агрегируют «формулы», создают виртуальную колонку
 - SUMX(Sales; Sales[Price] * Sales[Quantity])
 - AVERAGEX
 - MINX, MAXX
 - RANKX
 - COUNTAX, COUNTX и другие
- Итерируют по таблице, делают расчет для каждой строки, и уже затем агрегируют
- Всегда нужны 2 параметра
 - Таблица в которой будем итерировать (создавать виртуальную колонку)
 - Формула которую надо рассчитать для каждой строки, и уже затем агрегировать всю колонку

FILTER

- Добавляет условие фильтрации к таблице (WHERE)
- Сокращает кол-во строк в исходной таблице
- Возвращает «кусочек» исходной таблицы, этот кусочек затем можно использовать в X-функции
- Исходной таблицей может быть FILTER
- AND, OR, &&, ||

NB: PowerBI vs SQL

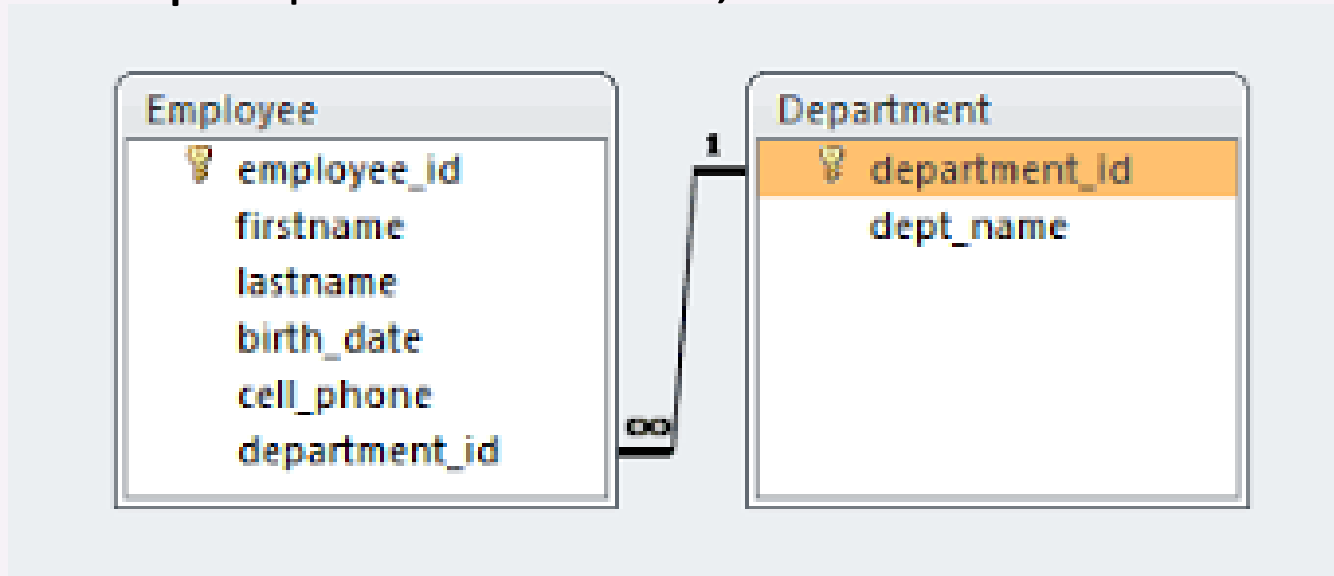


SQL

FILTER (DAX) = WHERE

RELATED, RELATEDTABLE

- **RELATED** – возвращает *значение*, по связи “Многие к *одному*”



- **RELATEDTABLE** – возвращает *таблицу*, по связи “Один ко *многим*”

Logical functions

- IF
- SWITCH

Variables

- Используйте переменные, чтобы
 - избежать повторения одинаковых выражений в расчетах, не создавать много мер/колонок
 - ускорить/оптимизировать расчеты (переменная рассчитывается только 1 раз)
 - запоминает контекст строки в котором переменная была вычислена

NewMeasure =

VAR

TotalQuantity = SUM (Sales[Quantity])

RETURN

IF (TotalQuantity > 1000;

TotalQuantity * 0.95;

TotalQuantity * 1.25)

EARLIER

- Работает с контекстом строки
- Возвращает значение столбца на предыдущих уровнях контекста строки

TIME INTELLIGENCE

- PREVIOUSMONTH
 - DATESADD
 - SAMEPERIODLASTYEAR
 - DATESYTD
-
- Нужен Календарь (справочник дат) для правильной работы
 - ВСЕ даты каждого года должны присутствовать в Календаре, иначе TI не будет корректно работать: с 1 января по 31 декабря (а у нас в календаре что?)

ALL – дЭрзкая функция

- Возвращает ВСЕ строки таблицы
- Игнорирует все фильтры (дэрзкая)
- Возвращает всю исходную таблицы, ее также затем можно использовать в X-функции
- Можно использовать для колонки ALL (Customers[CustomerName])
 - результат будет содержать таблицу из одной колонки всех уникальных значений исходной колонки

ALLEXCEPT – игнорирует все фильтры кроме тех столбцов, которые указаны как параметры этой функции (зеркало ALL(Столбец))

ALLSELECTED – удаляет последний примененный фильтр во всех таблицах, в конкретной таблице, или в конкретном столбце

ALL, ALLNOBLANKROW vs. VALUES, DISTINCT для фильтрации уникальных значений

- ALLNOBLANKROW – как ALL, только игнорирует пустые строки, возвращает ВСЕ НЕПУСТЫЕ строки таблицы
- VALUES – как ALL, возвращает пустые строки, только НЕ ДЭРЗСКАЯ, не игнорирует фильтры
- DISTINCT – как ALLNOBLANKROW, не возвращает пустые строки, только НЕ ДЭРЗСКАЯ, не игнорирует фильтры

RANKX

- Возвращает ранжирование числа в списке чисел для каждой строки в аргументе `table`.
- `RANKX(<table>, <expression>[, <value>[, <order>[, <ties>]]])`
 - Skip (RANK в SQL) - Следующее значение ранжирования — это значение ранга для связи, а также число привязанных значений. Например, если пять (5) значений связаны с рангом 11, то следующему значению будет присвоен 16 ($11 + 5$). Это значение по умолчанию, если параметр `ties` опущен.
 - Dense (DENSE_RANK в SQL) - Следующее значение ранга — это следующий ранг. Например, если пять (5) значений связаны с рангом 11, то следующее значение получит ранг 12.

ADDCOLUMNS()

- добавляет столбец к таблице и возвращает таблицу с добавленным столбцом
- часто используется в вычислениях, где нужно чтобы столбец был создан, но лишь виртуально, не занимая место в модели данных
- похожа на X-функции, и в случае когда надо создать 1 столбец и нужно сделать простую агрегацию, лучше и проще использовать X-функции, но когда надо создать более 1 столбца/сложные вычисления - ADDCOLUMNS

SUMMARIZE()

- создает сводную виртуальную таблицу со сгруппированными вычислениями/агрегатами (похожа на GROUP BY)

SELECTCOLUMNS

- выбор столбца из таблицы

SUMMARIZE()

- создает сводную виртуальную таблицу со сгруппированными вычислениями/агрегатами (похожа на GROUP BY)
- хорошо работает с небольшим объемом данных/небольшими исходными таблицами, но если в исходной таблице очень много данных, начинает «тормозить» >>> поэтому в случае большой исходной таблицы, лучше использовать комбинацию ADDCOLUMNS + SUMMARIZE + CALCULATE (но в этом случае не будет возможности создавать промежуточные итоги через ROLLUP/ROLLUPGROUP)

GROUPBY()

- создает сводную виртуальную таблицу со сгруппированными вычислениями/агрегатами
- нет возможности создавать промежуточные итоги (как SUMMARIZE через ROLLUP/ROLLUPGROUP)
- работает быстрее SUMMARIZE
- нужно использовать вместе с X-aggregation functions и CURRENTGROUP()

TOPN

- Создает таблицу из топ-N строк исходной таблицы

ABC анализ

— метод, позволяющий классифицировать ресурсы фирмы по степени их важности. Этот анализ является одним из методов рационализации и может применяться в сфере деятельности любого предприятия.

В его основе лежит **принцип Парето** — 20 % всех товаров дают 80 % оборота. По отношению к ABC-анализу правило Парето может прозвучать так: надёжный контроль 20 % позиций позволяет на 80 % контролировать систему, будь то запасы сырья и комплектующих, либо продуктовый ряд предприятия и т. п.

	X	Y	Z
A	AX	AY	AZ
B	BX	BY	BZ
C	CX	CY	CZ

Категория, на которую надо обратить внимание. Важные товары, но с абсолютно не стабильными продажами.

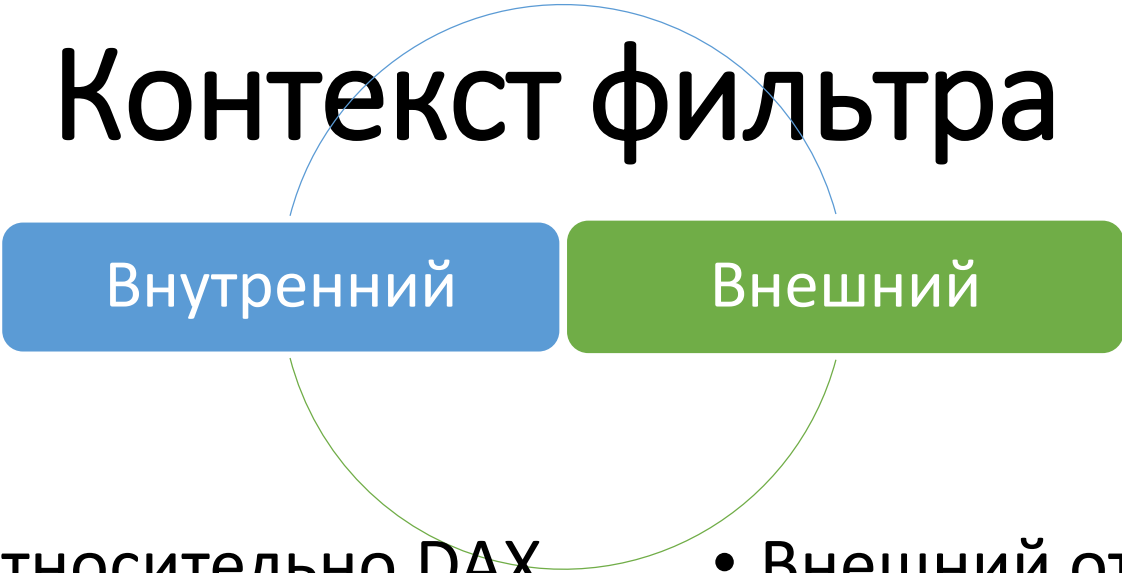
Товары обеспечивают основной товарооборот и стабильно продаются, поэтому необходимо обеспечивать постоянное их наличие.

Категория требует вдумчивого анализа. Эти товары можно выводить из ассортимента, если они не новые, элитные и т.п.

CALCULATE()

- меняет контекст фильтра
 - контекст строки превращает в контекст фильтра
 - а также, CALCULATE - это ВНУТРЕННИЙ контекст фильтра, который удаляет ВНЕШНИЙ контекст фильтра
-
- CALCULATE() – для скалярного выражения, CALCULATETABLE() – для табличных выражений, в остальном эти две функции одинаковые

Контекст фильтра



Внутренний

Внешний

- Внутренний относительно DAX
 - фильтры внутри DAX
формулы, прописанные
кодом
 - накладываются, используя
CALCULATE(),
CALCULATETABLE()
- Внешний относительно DAX
 - фильтры введенные
пользователем во вкладке
Report
 - Slicers, вкладка Filters,
строки/столбцы в
табличной визуализации, и
тп

Внутренний главнее, чем Внешний



CALCULATE – функция-исключение

сначала вычисляется 2 параметр (фильтр или фильтры), а затем первый (выражение)

Во всех других функциях DAX сначала вычисляется первый параметр, затем второй (вспомним FILTER, X-функции)

CALCULATE() - это **ВНУТРЕННИЙ** контекст фильтра, который главнее, чем **ВНЕШНИЙ** контекст фильтра и...

- дополняет внешний – если по разным столбцам
- заменяет внешний – если по одному и тому же столбцу
- удаляет внешний – с помощью ALL